

Regulatory Compliance Engine — Especificação Técnica

Motor de conformidade regulatória para tropicalização de serviços financeiros (EUA → Brasil)

Nota de originalidade: projeto **fictício e original**, desenvolvido do zero para fins de portfólio. Cenário, dados e regras são ilustrativos; não reproduz sistemas, código, arquitetura ou soluções proprietárias de nenhuma empresa. As tecnologias citadas (Go, Fiber, MongoDB, PHP) são de domínio público.

1. A ideia em uma frase

Assim como um produto importado passa pela **alfândega** antes de entrar no país, um **produto financeiro criado sob as regras dos EUA** precisa passar por uma verificação de conformidade antes de operar no Brasil. Este projeto é essa "alfândega regulatória": recebe uma operação ou a configuração de um produto e responde "**pode operar no Brasil? o que precisa ser adaptado?**", aplicando regras regulatórias brasileiras de forma configurável.

2. Contexto e problema

Empresas de tecnologia que atendem o setor financeiro convivem com um desafio recorrente: **traduzir norma regulatória em software**. Quando um serviço financeiro concebido no exterior (tipicamente nos EUA) é trazido para o Brasil — o processo de *tropicalização* —, ele precisa se adequar a exigências locais que não existiam na origem: tributação específica, limites, obrigações de comunicação a órgãos reguladores, prevenção à lavagem de dinheiro e proteção de dados.

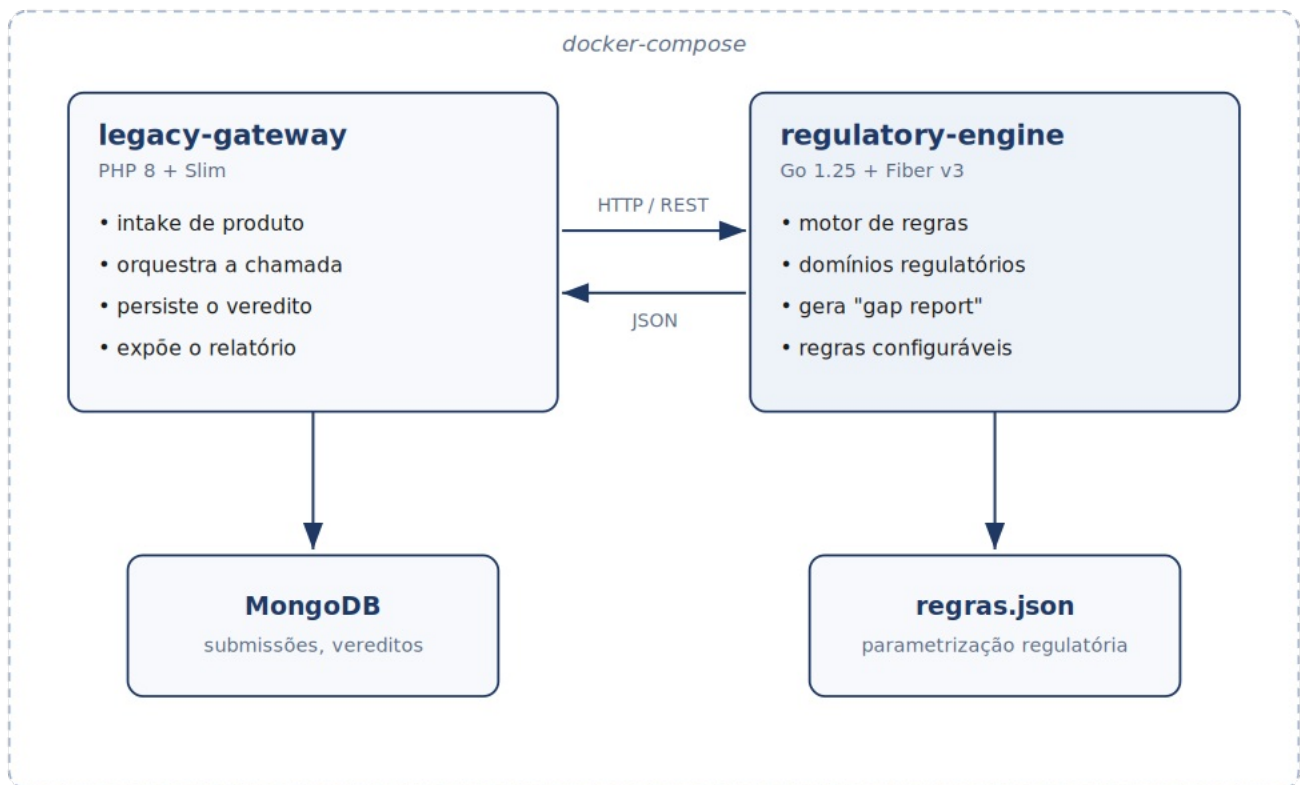
Fazer essa adequação manualmente, produto a produto, é lento e propenso a erro. Este projeto sistematiza a verificação: um serviço central que, dada uma operação, aponta objetivamente as exigências regulatórias brasileiras aplicáveis e as adaptações necessárias.

O projeto demonstra três capacidades:

1. **Entendimento do domínio** financeiro/regulatório brasileiro.
2. **Transformar regra de negócio em software** bem estruturado.
3. **Arquitetura poliglota** com integração entre um sistema legado e uma nova API.

3. Arquitetura

Dois microsserviços containerizados, orquestrados por Docker Compose, reproduzindo um padrão real de migração/modernização gradual — a coexistência entre um sistema legado e uma nova API:



Papéis:

- **regulatory-engine (Go + Fiber):** coração do projeto. Recebe uma operação/produto e devolve o veredito de conformidade. Contém o motor de regras e os domínios regulatórios. Não tem estado de negócio (stateless); lê a parametrização de um arquivo/tabela de regras.
- **legacy-gateway (PHP + Slim):** representa o sistema legado de uma instituição financeira. Faz o *intake* das submissões, chama o motor Go, persiste o resultado no MongoDB e expõe o histórico e o relatório regulatório. É o ponto de integração legado ↔ moderno.

Decisão de arquitetura-chave: as regras (limites, alíquotas, gatilhos de reporte) **não são hardcoded** — vivem em parametrização configurável (`rules.json` , evoluível para tabela no banco). É uma boa prática essencial no contexto regulatório, porque norma muda com frequência.

4. Domínios regulatórios (escopo inicial)

Sobre os valores: os números abaixo são *ilustrativos* para a estrutura funcionar. Antes de finalizar, confirmar os parâmetros vigentes nas fontes oficiais (Bacen, COAF/UIF, Receita Federal) e ajustar no `rules.json` . A arquitetura configurável existe justamente para isso.

4.1. PLD/FT + COAF (Prevenção à Lavagem de Dinheiro)

- Detecção de operações acima de limite de reporte.
- Marcação de operações **reportáveis ao COAF** (ex.: espécie acima de teto, fracionamento suspeito).
- Screening simples contra lista de **PEP / sancionados** (lista mock local).
- Saída: alertas + indicação de comunicação obrigatória.

4.2. Câmbio / IOF (o core da "tropicalização")

- Cálculo de **IOF** na entrada de recursos EUA → BR.
- Conversão cambial (taxa parametrizável / mock).
- Regras específicas por tipo de operação (transferência, investimento, empréstimo).
- Saída: tributos aplicáveis + adaptações necessárias vs. o produto original.

4.3. (Roadmap) Limites Bacen/Pix + LGPD

- Fica documentado como próxima fase, para o projeto ter história de evolução.

5. Contrato da API (regulatory-engine)

```
POST /api/v1/evaluate
→ avalia um produto/operação, PERSISTE e retorna a avaliação criada (201)
  { id, created_at, request, report }
GET /api/v1/evaluations
→ lista as avaliações mais recentes
GET /api/v1/evaluations/{id}
→ recupera uma avaliação pelo id
GET /api/v1/health
→ healthcheck
GET /api/v1/rules
→ lista as regras carregadas (bloco futuro)
GET /swagger/*
→ documentação OpenAPI (bloco futuro)
```

Exemplo de requisição (`POST /api/v1/evaluate`):

```
{
  "product": {
    "type": "personal_loan",
    "origin": "US",
    "origin_currency": "USD"
  },
  "operation": {
    "amount": 75000.00,
    "currency": "USD",
    "method": "international_transfer",
    "counterparty": { "name": "John Doe", "pep": false }
  }
}
```

Exemplo do report (dentro da avaliação retornada pelo `POST`):

```
{
  "compliant": false,
  "results": [
    {
      "domain": "IOF",
      "status": "adaptation_required",
      "detail": "Entrada convertida de USD para BRL a 5.00; IOF de 0.38% aplicável (produto original não previa tributação).",
      "calculated_amount": "1425.00",
      "reference": "rule:iof.international_transfer"
    },
    {
      "domain": "PLD_COAF",
      "status": "reportable",
      "detail": "Valor acima do teto de comunicação automática ao COAF.",
      "reference": "rule:pld.reporting_threshold"
    }
  ],
  "required_adaptations": [
    "Incluir cálculo e retenção de IOF no fluxo de entrada.",
    "Gerar comunicação ao COAF para a operação."
  ]
}
```

Valores monetários trafegam como **string com 2 casas** ("1425.00"), para não perder precisão no cliente. Na entrada, o campo `amount` aceita número ou string. Alíquotas e câmbio (parametrização) usam decimal simples.

6. Estrutura de pastas (monorepo)

```

regulatory-compliance-engine/
├── docker-compose.yml
├── Makefile # atalhos de desenvolvimento
├── README.md
├── docs/
│   ├── especificacao_tecnica.md
│   └── arquitetura.png # diagrama (fonte: arquitetura.svg)
├── regulatory-engine/ # Go + Fiber
│   ├── cmd/api/main.go # ponto de entrada (montagem da aplicação)
│   ├── internal/
│   │   ├── config/ # configuração via ambiente
│   │   ├── database/ # conexão com o MongoDB
│   │   ├── model/ # structs do domínio (contrato)
│   │   ├── money/ # tipo monetário (decimal, sempre 2 casas)
│   │   ├── engine/ # interface Rule + avaliador
│   │   ├── rules/ # regras (ex.: IOF) + carregamento do rules.json
│   │   └── httpapi/ # servidor, rotas e handlers
│   ├── config/rules.json # parametrização regulatória
│   ├── go.mod
│   └── Dockerfile # (bloco futuro)
├── legacy-gateway/ # PHP + Slim (bloco futuro)
│   ├── public/index.php
│   ├── src/
│   │   ├── Controller/
│   │   ├── Service/EngineClient.php # integração com o motor Go
│   │   └── Repository/
│   ├── composer.json
│   └── Dockerfile
├── db/ # seed de coleções/índices (bloco futuro)
└── init.js

```

7. Modelo de dados (MongoDB)

Banco `regulatory`. A avaliação é gravada como **um único documento embedded** na coleção `evaluations` — o modelo idiomático de MongoDB: uma escrita, e uma leitura traz o quadro completo, sem "join".

- **evaluations**: { `_id`, `created_at`, `request` { ... }, `report` { ... } } — a submissão original e o veredito produzido, juntos no mesmo documento.

Valores monetários são gravados como **Decimal128** (o decimal nativo do Mongo), preservando a precisão e permitindo consultas e agregações por valor.

Decisão de modelagem (embedded vs. referência). `request` e `report` têm relação 1:1, nascem juntos e são sempre lidos juntos — portanto ficam **embutidos** no mesmo documento (separá-los em duas coleções seria um anti-padrão: duas escritas não-atômicas e um *join* na leitura, sem benefício). Como a avaliação é um **registro de auditoria**, o documento é tratado como um **retrato imutável** do que foi avaliado e do veredito daquele momento. Promover a **contraparte** a entidade própria (coleção `parties`) só se justificaria com uma **identidade estável** — um documento (CPF/CNPJ), não o nome — e fica no roadmap.

8. Testes e qualidade

A garantia de qualidade do `regulatory-engine` se apoia em três pilares implementados: uma suíte de testes automatizados, um pipeline de integração contínua que a executa a cada mudança, e logs estruturados para observabilidade.

8.1. Estratégia de testes

A suíte usa `testify` e cobre as três camadas do motor de forma independente:

- **Testes unitários (table-driven).** As regras de negócio — cálculo de IOF, teto de comunicação ao COAF, screening de PEP/sancionados — e o tipo monetário são testados com tabelas de casos (entrada esperada vs. saída), o padrão idiomático em Go. Cada regra é verificada isoladamente: quando se aplica, o valor calculado, e quando **não** se aplica (retorno nulo).
- **Teste de agregação do motor com duplê.** O avaliador (`engine`) é testado contra uma **regra falsa** (`fakeRule`) controlada pelo teste, não contra as regras reais. Assim se verifica apenas a responsabilidade do

motor — agregar resultados, ignorar regras que não se aplicam, decidir conformidade e propagar erro — sem acoplamento ao comportamento de IOF ou PLD.

- **Teste de handler HTTP sem banco.** Os handlers dependem da **interface** `EvaluationStore`, não do repositório concreto. Nos testes, injeta-se um **store falso em memória** (`fakeStore`) e exercitam-se as rotas de ponta a ponta com `app.Test` (sem abrir porta de rede nem exigir MongoDB): `POST /evaluate` retornando `201` e persistindo, corpo inválido retornando `400`, e busca inexistente retornando `404`.

O ponto de projeto que torna isso possível é a **injeção de dependência** adotada nos blocos anteriores: como o servidor recebe o motor e o store por interface, ambos podem ser substituídos por dublês nos testes. Testes rápidos, determinísticos e que rodam em qualquer máquina limpa — inclusive no CI, sem infraestrutura.

8.2. Integração contínua (CI)

Um workflow de **GitHub Actions** roda a cada `push` na `main` e em todo `pull request`, numa máquina limpa do runner. A sequência reproduz a verificação local:

1. `gofmt` — falha o build se houver código fora do padrão de formatação da linguagem.
2. `go build ./...` — garante que todo o módulo compila.
3. `go vet ./...` — análise estática de problemas comuns.
4. `go test ./...` — executa a suíte descrita acima.

O ambiente é fixado em Go 1.25 com cache de módulos. O valor concreto: a verificação deixa de depender da disciplina manual do desenvolvedor — um arquivo esquecido no commit, um `go.sum` inconsistente ou código desformatado são barrados antes de entrar na `main`. O estado do pipeline é exposto por um *badge* no README.

8.3. Observabilidade

Os dois serviços emitem **logs estruturados em JSON** (via `slog` no motor Go). Um *middleware* de requisição registra método, rota, status e latência de cada chamada em formato de campo — pronto para ser filtrado por um agregador (Loki, ELK, CloudWatch) sem *parsing* de texto livre.

8.4. Demonstração

- **README** com contexto de negócio, diagrama, como rodar (`docker compose up`), exemplos de chamada e o **roadmap**.
- **Coleção de exemplos** (curl / Postman) para rodar em poucos minutos.

9. Roadmap (evolução futura)

1. **Gateway PHP + Slim**: serviço que representa o sistema legado e chama o motor Go (integração legado ↔ novo).
2. **Coleção parties (identidade de contraparte)**: promover a contraparte a entidade de primeira classe, identificada por **documento (CPF/CNPJ/tax id)** — não pelo nome — com índice único, e screening por documento. Envolve *entity resolution* (fuzzy matching contra listas de sanção), um problema à parte.
3. Domínios adicionais: Limites Bacen/Pix, LGPD, SCR.
4. Observabilidade completa: Prometheus + Grafana + Loki.
5. Regras em banco com versionamento (histórico de vigência das normas).
6. Screening PEP/sancionados contra fonte real.
7. Autenticação (JWT) e trilha de auditoria.